Informe: reparto de presupuesto y selección de participantes en una red

1. Introducción

El proyecto estudia dos problemas relacionados con la selección de nodos de una red:

1. **Reparto de presupuesto:** distribuir un presupuesto entero entre nodos con utilidades crecientes y cóncavas.
2. **Selección de participantes:** escoger como máximo k nodos para maximizar una utilidad de cobertura.

El primer problema admite una solución exacta eficiente. El segundo contiene como caso particular a Maximum Coverage, por lo que su versión de optimización es NP-difícil. Para este último se implementan una referencia exacta por fuerza bruta y un algoritmo Greedy con garantía de aproximación bajo las hipótesis indicadas.

2. Parte I: reparto de presupuesto

2.1. Entrada, salida y restricciones

La entrada consiste en:

- N nodos.
- Un presupuesto entero $B \geq 0$.
- Para cada nodo i, una secuencia de ganancias marginales $\Delta_i(1), \ldots, \Delta_i(m_i)$.

La utilidad de asignar x_i unidades al nodo i es:

> $u_i(x_i) = \sum_{j=1..x_i} \Delta_i(j)$

Se exige:

- $0 \leq x_i \leq m_i$
- $\sum_i x_i \leq B$
- $\Delta_i(j+1) \leq \Delta_i(j)$ para toda posición válida; esta es la concavidad discreta.
- Las ganancias deben ser finitas y no negativas.

La salida es una asignación $(x_1, \ldots, x_N)$ y la utilidad total $\sum_i u_i(x_i)$.

2.2. Algoritmo exacto

Se inserta en una cola de prioridad la primera marginal disponible de cada nodo. En cada iteración se extrae la mayor marginal, se asigna esa unidad y se inserta la siguiente marginal del mismo nodo. El procedimiento termina al usar B unidades o cuando no quedan unidades disponibles.

La implementación se encuentra en `red_project/allocation.py`, función `greedy_allocate`.

2.3. Corrección

Considérese una solución factible que contiene una unidad con marginal a, mientras existe una unidad disponible con marginal $b \geq a$. Intercambiar la unidad de marginal a por la de marginal b no disminuye la utilidad. Repitiendo este intercambio se obtiene una solución formada por las mayores marginales factibles.

La concavidad garantiza que las marginales de un nodo aparecen en orden no creciente. Por tanto, si una marginal posterior está disponible, todas sus predecesoras ya fueron elegidas. La cola de prioridad siempre representa exactamente la siguiente elección factible de cada nodo. En consecuencia, cada extracción es segura y el algoritmo termina con una solución óptima.

2.4. Complejidad

La inicialización de la cola cuesta $O(N)$. Se realizan como máximo B extracciones y cada extracción, junto con una posible inserción, cuesta $O(\log N)$. Por tanto:

> $$T(N, B) = O(N + B \log N)$$

El espacio adicional es $O(N)$, sin contar las marginales de entrada.

3. Parte II: selección de participantes

3.1. Entrada, salida y restricciones

La instancia de cobertura está formada por:

- Un universo U de elementos.
- N nodos, donde cada nodo i cubre un subconjunto $C_i \subseteq U$.
- Un límite de cardinalidad k, con $0 \leq k \leq N$.

La utilidad de una selección S es:

> $$f(S) = \left| \bigcup_{i \in S} C_i \right|$$

La salida es un subconjunto S con $|S| \leq k$ que maximiza $f(S)$. La implementación trabaja con selecciones de tamaño k cuando $k > 0$; para $k = 0$ devuelve el conjunto vacío.

La función es:

- **No negativa:** $f(S) \geq 0$.
- **Monótona:** si $A \subseteq B$, entonces $f(A) \leq f(B)$.
- **Submodular:** si $A \subseteq B$ y $i \notin B$, entonces $f(A \cup \{i\}) - f(A) \geq f(B \cup \{i\}) - f(B)$.

La submodularidad expresa rendimientos decrecientes: un nodo cubre menos elementos nuevos cuando ya se han seleccionado muchos nodos.

3.2. Algoritmos implementados

Referencia exacta: `brute_force_select` enumera todas las combinaciones de k nodos y conserva la de mayor utilidad. Su complejidad es $O(C(N, k) \cdot c_f)$, donde c_f es el costo de evaluar la utilidad. Para cobertura explícita, c_f depende del número de elementos cubiertos. Este algoritmo solo debe usarse en instancias pequeñas.

Greedy por ganancia marginal: `greedy_select` comienza con $S = \emptyset$. En cada paso evalúa cada nodo no seleccionado, calcula su ganancia marginal y agrega el nodo con mayor ganancia. Realiza a lo sumo $N \cdot k$ evaluaciones de la función, por lo que su costo es $O(N \cdot k \cdot c_f)$.

Para una función no negativa, monótona y submodular con restricción de cardinalidad, el teorema de Nemhauser, Wolsey y Fisher establece:

> $f(S_{\text{greedy}}) \geq (1 - 1/e)\, f(S^*) \approx 0.632\, f(S^*)$

La cota es un límite inferior teórico, no una igualdad. Las instancias concretas pueden producir una razón mayor.

4. Prueba formal de NP-dureza por reducción polinomial

Se demuestra la NP-dureza de la selección general mediante una reducción desde Maximum Coverage, conocido como problema NP-difícil.

4.1. Problema de partida

Una instancia de Maximum Coverage contiene un universo U, una colección de subconjuntos $C_1, \ldots, C_N$ y un entero k. Se busca seleccionar como máximo k subconjuntos que cubran el mayor número de elementos. Su versión de decisión pregunta si existe una selección de a lo sumo k subconjuntos cuya cobertura tenga tamaño al menos q.

4.2. Transformación

Dada una instancia de Maximum Coverage, se construye una instancia de selección de la siguiente forma:

1. Crear un nodo v_i por cada subconjunto C_i.
2. Asociar al nodo v_i exactamente el conjunto C_i.
3. Mantener el mismo valor k.
4. Definir la utilidad como $f(S) = |\bigcup_{v_i \in S} C_i|$.
5. En la versión de decisión, mantener también el umbral q.

La transformación solo copia los conjuntos y sus índices. Su tiempo es $O(N + \sum_i |C_i|)$ y su tamaño es polinómico respecto de la entrada.

4.3. Preservación de la respuesta

Para toda selección $I \subseteq \{1, \ldots, N\}$:

> $|\bigcup_{i \in I} C_i| = f(\{v_i : i \in I\})$

Por lo tanto, una solución de Maximum Coverage de valor al menos q existe si y solo si existe una selección de nodos de valor al menos q. La respuesta se conserva en ambas direcciones.

Si existiera un algoritmo polinomial que resolviera óptimamente la selección general, podríamos transformar y resolver cualquier instancia de Maximum Coverage en tiempo polinomial. Esto implicaría P = NP. En consecuencia, la selección general es NP-difícil.

Además, la versión de decisión pertenece a NP: dado un conjunto de nodos, se puede verificar en tiempo polinomial que tiene tamaño permitido y que su cobertura alcanza q. Por ello, la versión de decisión de este caso de selección es NP-completa.

La conclusión no se aplica al reparto cóncavo: ese problema específico sí se resuelve en tiempo polinomial mediante `greedy_allocate`.

5. Variantes

5.1. Restricción de presupuesto monetario

`greedy_budgeted_select` recibe costos positivos y un presupuesto real. Escoge por densidad de ganancia marginal (ganancia/costo) y compara la solución con el mejor nodo individual factible. Esta salvaguarda mejora el comportamiento práctico, pero la cota $1 - 1/e$ de cardinalidad no debe trasladarse sin más: la restricción de costos requiere el teorema específico de cobertura con presupuesto y sus hipótesis.

5.2. Fallos entre rondas

`select_over_rounds` ejecuta una selección Greedy por ronda y retira nodos según una probabilidad de fallo. La semilla permite reproducir la secuencia de fallos. La utilidad total es la suma de las utilidades de las rondas.

5.3. Externalidades de vecindad

`neighborhood_utility` suma a la utilidad base una influencia por los vecinos afectados. Dependiendo de la estructura de los vecindarios, la función resultante puede no conservar monotonía o submodularidad. Por ello esta variante se presenta como modelo experimental y no recibe automáticamente la garantía de Greedy.

6. Validación experimental

El módulo `red_project/experiments.py` genera instancias deterministas de Maximum Coverage, mide Greedy y fuerza bruta, y guarda:

- `results/times.csv`: resultados de cada repetición y semilla.
- `results/summary.csv`: promedios y desviaciones estándar.
- `results/times.png`: comparación de tiempos.
- `results/approximation.png`: razón Greedy/óptimo.

La fuerza bruta crece combinatoriamente, por lo que los tamaños experimentales deben mantenerse pequeños. La comparación empírica sirve para observar el crecimiento y la calidad, pero no sustituye la demostración formal.

7. Referencia bibliográfica

G. L. Nemhauser, L. A. Wolsey y M. L. Fisher, "An analysis of approximations for maximizing submodular set functions-I", *Mathematical Programming*, 14, 1978, 265-294.